



Experiment plan: minimum SAE circuit discovery



Goal of Paper 1: discover the minimum subset of SAE features that causally implements a target language-model behavior, and benchmark search/optimization methods against the 5 leading baselines on a Pareto frontier of *circuit size* $\times$ *faithfulness*.

Hardware: Colab Pro+ (A100 40GB or V100 16GB). **Model:** GPT-2-small. **SAE:** pre-trained `gpt2-small-res-jb` (no SAE training needed).

Target venue: TMLR (primary) or BlackBoxNLP 2026 (faster).

Repos to merge

Repo	Role	Why this one
TransformerLensOrg/TransformerLens	Model loading + activation hooks	Standard mech-interp library; the language all others speak
decoderesearch/SAELens (Joseph Bloom)	Pre-trained SAE loading	Has GPT-2-small residual-stream SAEs ready; no training needed
saprmrks/feature-circuits	Reference SAE-feature circuit discovery (Marks et al., ICLR 2025)	The paper Problem 1 directly extends — most important starting point
saprmrks/dictionary_learning	SAE training (only if needed later)	Skip for Paper 1
ArthurConmy/Automatic-Circuit-Discovery	ACDC baseline (head-level)	Reference baseline #1
hannamw/eap-ig	EAP-IG: fast $O(m)$ circuit discovery (COLM 2024)	$\sim 100\times$ faster than ACDC; drop-in

Repo	Role	Why this one
		replacement
hannamw/MIB-circuit-track	Unified harness: EAP-IG-inputs/activations, exact activation patching, Information-Flow Routes	Use this — gives you 4 baselines + faithfulness metric for free
hannamw/eap-ig-faithfulness	Faithfulness evaluation code	Plug into eval loop
Bussmann et al. binary-mask circuit finding	Continuous binary mask + Gumbel-Sigmoid; 95% faithfulness with <20 latents on Gemma-9B	Your nearest competitor — read carefully



Your contribution sits in the gap between Marks et al. (greedy / attribution-based) and Bussmann et al. (binary-mask continuous relaxation): **discrete search/optimization-based minimum-circuit discovery with explicit NFL-style comparison of search strategies** (PSO, GA, SA, ACO).

Pipeline architecture

```
graph TD
    A["GPT-2-small (TransformerLens)"] --> B["Pre-trained SAEs (SAELens: gpt2-small-res-jb)"]
    B --> C["IOI task dataset (1000 prompts, clean+corrupt)"]
    C --> D["Cache SAE feature activations (one-time)"]
    D --> E["Circuit Discovery Methods"]
    E --> E1["Baseline 1: Magnitude"]
    E --> E2["Baseline 2: EAP"]
    E --> E3["Baseline 3: EAP-IG"]
    E --> E4["Baseline 4: Greedy (Marks et al.)"]
    E --> E5["Baseline 5: Binary mask (Bussmann et al.)"]
    E --> E6["Your method: search-based (PSO / GA / SA / ACO)"]
    E1 --> F["Faithfulness eval (CE / logit-diff recover
```

y)"]

E2 --> F

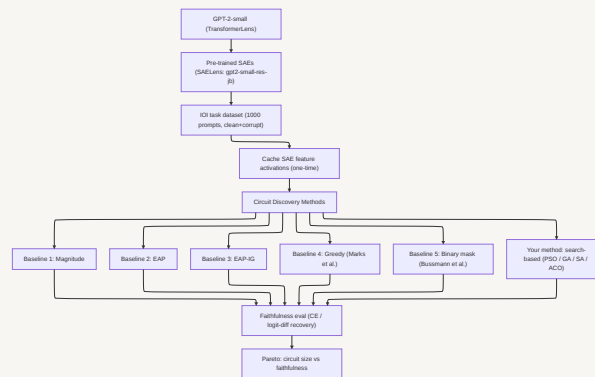
E3 --> F

E4 --> F

E5 --> F

E6 --> F

F --> G["Pareto: circuit size vs faithfulness"]



8 tasks

Task 1 — Environment + replication

- ☐ Install stack: `transformer_lens`, `sae_lens`, `nnsight`, `datasets`
- ☐ Clone `feature-circuits`, `eap-ig`, `MIB-circuit-track`
- ☐ Load GPT-2-small + `gpt2-small-res-jb` layer 8 SAE
- ☐ Reproduce one feature-ablation result from Marks et al.

Deliverable: screenshot of one feature ablation that breaks IOI accuracy.

Task 2 — IOI dataset + baseline metrics

- ☐ Build IOI dataset (1000 prompts) using `transformer_lens.utils.IOIDataset` or ACDC repo data
- ☐ Implement two metrics: **faithfulness** = $\text{logit-diff}(\text{circuit}) / \text{logit-diff}(\text{full})$; **size** = active SAE features
- ☐ Sweep magnitude thresholds → first Pareto curve

Deliverable: Pareto plot with magnitude baseline (your x/y axes locked in).

Task 3 — Wire up 5 baselines

☐ Adapt `MIB-circuit-track` from heads → SAE features (point hooks at SAE latent dims; `feature-circuits/attribution.py` shows how)

☐ Run all 5 baselines on IOI

☐ Overlay all on the Pareto plot

Deliverable: Pareto plot with 5 baselines overlaid — already publishable as a comparison.

Task 4 — Implement your search-based method

Pick ONE search strategy first:

Method	Why try it	Difficulty
Greedy backward elimination	Strong classical baseline	Easy
PSO on binary subset ★	Direct connection to swarm-PhD background; novel for SAEs	Medium
Genetic algorithm	Population of subsets; swarm-PhD pattern	Medium
Simulated annealing	Escapes local minima; one-line addition to GA	Easy
Binary mask + Gumbel-Sigmoid	Differentiable; replicates Bussmann	Medium
Ant Colony Optimization on feature graph	Novel for this domain — nobody has tried	Medium

Recommendation: start with **PSO on binary subset representation**. Fitness function:

$$\text{fitness}(S) = \text{faithfulness}(S) - \lambda \cdot \frac{|S|}{N_{\text{total}}}$$

Sweep λ to generate the Pareto curve.

Deliverable: PSO results on the 5-baseline Pareto plot. If on or above Pareto frontier at any point → paper.

Task 5 — Robustness

☐ **Layers:** repeat on layers 6, 7, 8, 9 of GPT-2-small

☐ **Tasks:** IOI, Greater-Than, Gender Bias, Subject-Verb Agreement (all in `feature-circuits` repo)

☐ **SAE widths:** 24k vs 32k features

☐ **Seeds:** 5 runs per config, mean $\pm$ std

Deliverable: 4-task $\times$ 4-layer $\times$ 6-method results matrix $\rightarrow$ Table 1.

Task 6 — NFL terrain map (your differentiator)

☐ Show which method wins on which task $\times$ layer $\times$ budget cell

☐ Cluster tasks by dominant method

☐ Propose structural reasons (e.g., "greedy wins on sparse single-token tasks, PSO wins on multi-token interaction tasks")

Deliverable: per-cell winner heatmap. **This converts the paper from "yet another method" to "characterization of when each method works."**

Task 7 — Scaling (optional but strong)

☐ Repeat on Pythia-160M with SAELens `pythia-70m-deduped-res-sm`

☐ One experiment on Gemma-2-2B with Gemma Scope SAEs

Deliverable: scaling table — one Gemma row is enough.

Task 8 — Writing

☐ Structure: Intro $\rightarrow$ SAE circuit discovery as combinatorial search $\rightarrow$ 6 methods $\rightarrow$ multi-task results $\rightarrow$ NFL analysis $\rightarrow$ conclusion

☐ Target: **TMLR** (rolling, 8–12 pages) or **BlackBoxNLP 2026** (workshop)

☐ Draft, internal review, submit

Starter notebook (Task 1)

```
# === Setup ===
!pip install -q transformer_lens sae_lens nnsight
import torch, einops
from transformer_lens import HookedTransformer
from sae_lens import SAE
from functools import partial
```

```

device = "cuda"

# === Load model + SAE ===
model = HookedTransformer.from_pretrained("gpt2-small", device=device)
sae, cfg_dict, sparsity = SAE.from_pretrained(
    release="gpt2-small-res-jb",
    sae_id="blocks.8.hook_resid_pre",
    device=device,
)
sae.eval()

# === IOI prompts (build ~200 with this template) ===
ioi_prompts = [
    {"clean": "When John and Mary went to the store, John gave a book to",
     "corrupt": "When Alice and Bob went to the store, Alice gave a book to",
     "answer_clean": " Mary", "answer_corrupt": " Bob"},
    # ... add more
]

def logit_diff(logits, clean_tok, corrupt_tok):
    return (logits[..., clean_tok] - logits[..., corrupt_tok]).mean()

# === SAE-replacement hook ===
def sae_replace_hook(activation, hook, mask=None):
    """Replace residual stream with SAE reconstruction, optionally masking features."""
    feats = sae.encode(activation)
    if mask is not None:
        feats = feats * mask # binary: 1 = keep, 0 = ablate
    return sae.decode(feats)

# === Faithfulness function ===

```

```

def faithfulness(mask, prompts):
    full_diffs, masked_diffs = [], []
    for p in prompts:
        clean_tok = model.to_single_token(p["answer_clean"])
        corrupt_tok = model.to_single_token(p["answer_corrupt"])
        logits_full = model(p["clean"])[0, -1]
        full_diffs.append(logit_diff(logits_full, clean_tok, corrupt_tok))
        with model.hooks(fwd_hooks=[("blocks.8.hook_resid_pre",
                                     partial(sae_replace_hook, mask=mask))]):
            logits_masked = model(p["clean"])[0, -1]
            masked_diffs.append(logit_diff(logits_masked, clean_tok, corrupt_tok))
    return (torch.stack(masked_diffs).mean() / torch.stack(full_diffs).mean()).item()

# === Magnitude baseline ===
# Cache SAE feature activations across IOI prompts (one-time)
all_feats = []
for p in ioi_prompts:
    _, cache = model.run_with_cache(p["clean"], names_filter="blocks.8.hook_resid_pre")
    acts = sae.encode(cache["blocks.8.hook_resid_pre"])
    all_feats.append(acts.abs().mean(dim=(0, 1)))
mean_acts = torch.stack(all_feats).mean(0)
top_features = mean_acts.argsort(descending=True)

# === Sweep K and plot Pareto ===
import matplotlib.pyplot as plt
sizes, scores = [], []
for K in [5, 10, 20, 50, 100, 200, 500, 1000]:
    mask = torch.zeros(sae.cfg.d_sae, device=device)
    mask[top_features[:K]] = 1

```

```
sizes.append(K)
scores.append(faithfulness(mask, ioi_prompts))
plt.plot(sizes, scores, 'o-')
plt.xlabel("Circuit size"); plt.ylabel("Faithfulness"); pl
t.xscale("log")
plt.savefig("magnitude_pareto.png")
```

Pitfalls (these will kill weeks if unknown)



1. Patching subspaces $\neq$ patching features. Makelov et al. showed SAE-feature activation patching can create *interpretability illusions* — dormant pathways activate when you ablate. Always test with **mean-ablation + zero-ablation + resample-ablation** and report all three. This is the #1 reviewer complaint.



2. Don't ablate the SAE itself. Baseline = model *without* SAE substitution. Circuit = model *with* SAE substitution + mask. Otherwise SAE reconstruction error conflates with circuit incompleteness.



3. Logit-diff is on the last token, not the answer token position. Verify against the reference notebook.



4. Population methods need ≥ 5 seeds. PSO/GA/SA are stochastic. Single-seed results will fool you.



5. Don't train SAEs. `gpt2-small-res-jb` works great. Save weeks.



6. Cache activations to Google Drive once. SAE-encoding 1000 prompts $\times$ 12 layers takes ~ 1 min first time. Save to `.pt`. Saves ~ 10 min per iteration over the project.



7. Watch Colab Pro+ idle timeout (12h with background execution enabled). Use checkpoint-and-resume for long runs.

Reading priorities (do these before coding)

1. **Bussmann et al. "Scaling Sparse Feature Circuit Finding to Gemma 9B"** — your nearest competitor; binary mask + Gumbel-Sigmoid, 95% faithfulness with <20 latents.
2. **Marks et al. Sparse Feature Circuits, §3 only** — defines the SAE-feature circuit primitive.
3. **hannamw/MIB-circuit-track README** — the benchmark harness you'll plug into.

Summary card

Component	Choice	Why
Model	GPT-2-small	Standard; fast; baselines published on it
SAE	<code>gpt2-small-res-jb</code> layer 8	Pre-trained; ~25k features
Tasks	IOI + Greater-Than + Gender + Subj-Verb	Standard mech-interp suite
Eval metric	Faithfulness (logit-diff ratio) + size	Pareto curves
Baselines	Magnitude, EAP, EAP-IG, Greedy (Marks), Binary mask (Bussmann)	5 strong; all coded
Your method	PSO on binary subset (Task 4); +GA, SA (Task 5)	Swarm-PhD background; novel for SAEs
Differentiator	Multi-task NFL terrain map (Task 6)	Reviewers reward this framing
Stack	TransformerLens + SAELens + MIB-circuit-track + your code	Min boilerplate, max reuse
Compute	Colab Pro+ A100, 12h sessions	Sufficient for all experiments
Venue	TMLR (primary) / BlackBoxNLP 2026	Both publish work like this
Timeline	8 tasks to draft	Realistic given pre-trained SAEs + ready baselines